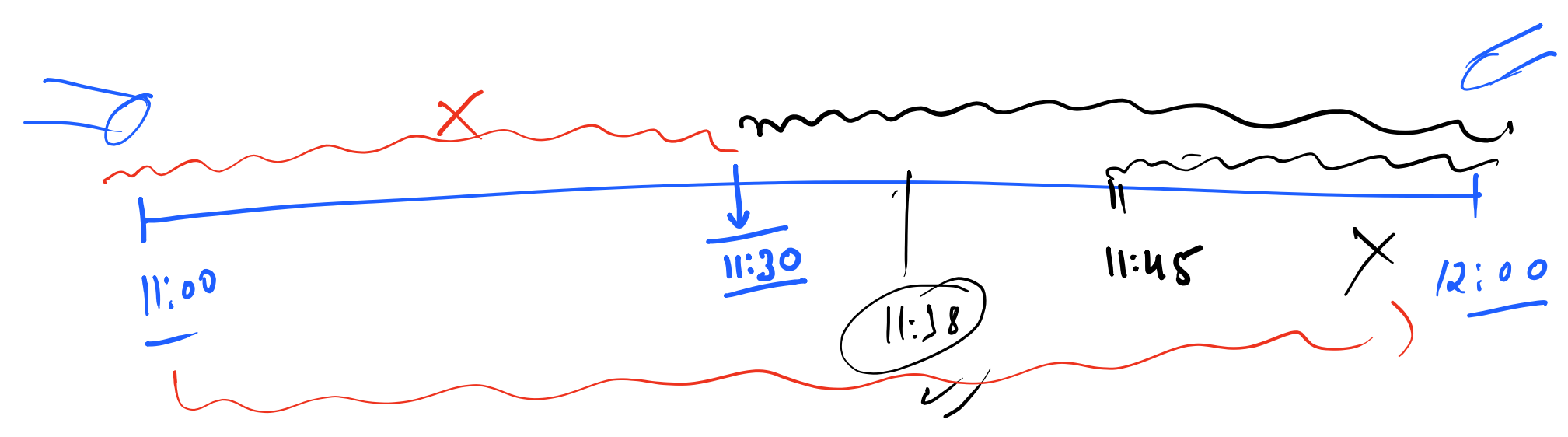
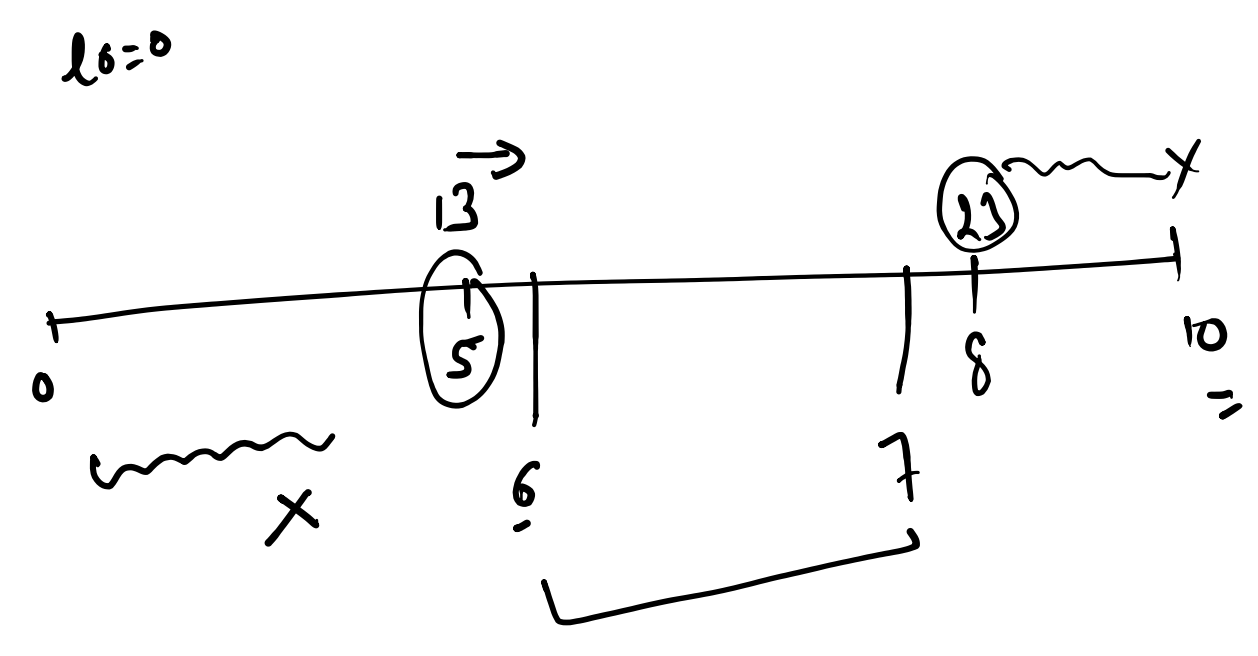
Lec-9

20/06/26 12:45 PM

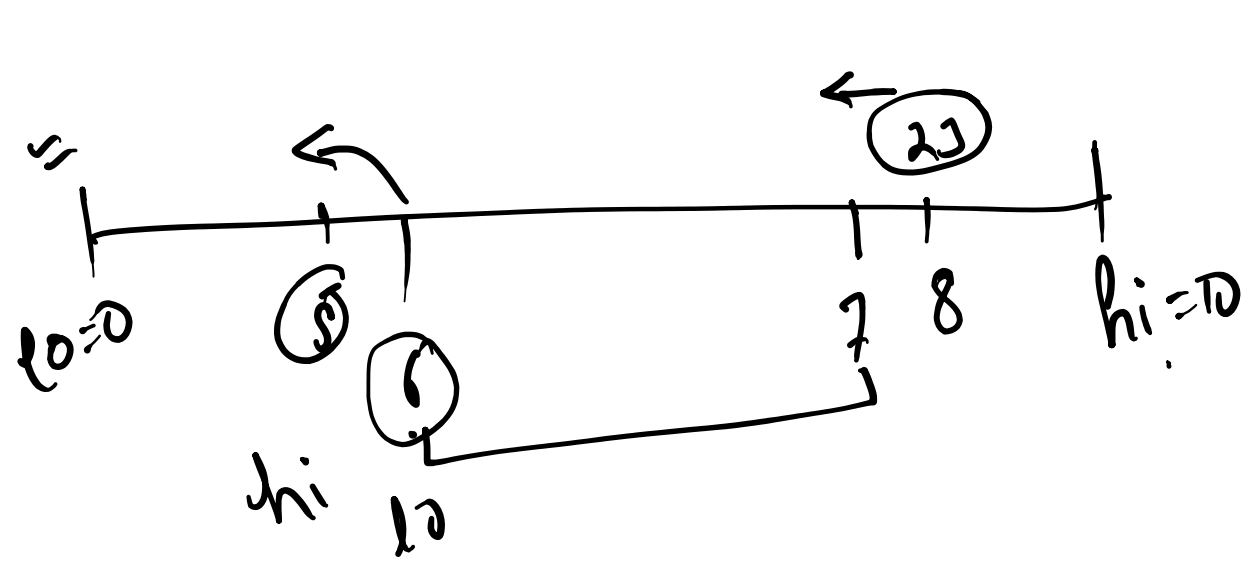

11:00
11:30
11:38
11:45
12:00

item=19

[2, 3, 5, 7, 11, 13, 19, 24, 23, 25, 27]
0 1 2 3 4 5 6 7 8 9 10
lo=0 hi=10


13
23
0
5
6
7
8
10

[2, 3, 5, 7, 11, 13, 19, 24, 23, 25, 27]
0 1 2 3 4 5 6 7 8 9 10
lo=0 hi=10


23
lo=0
5
6
7 8
hi=10
hi
lo

```
while ( lo <= hi ) {
    mid = (lo+hi)/2
    if ( arr[mid] == item ) {
        return mid
    }
    else if ( arr[mid] > item ) {
        hi = mid-1
    }
    else {
        lo = mid+1
    }
}
```

$x^k <= N$

K=v
N=v

K=3
N=149

x -> maximum

x=1
x=2
x=3
x=4
x=5

$1^3 \le 149$
$2^3 \le 149$
$3^3 \le 149$
$4^3 \le 149$
$5^3 \le 149$

x=5

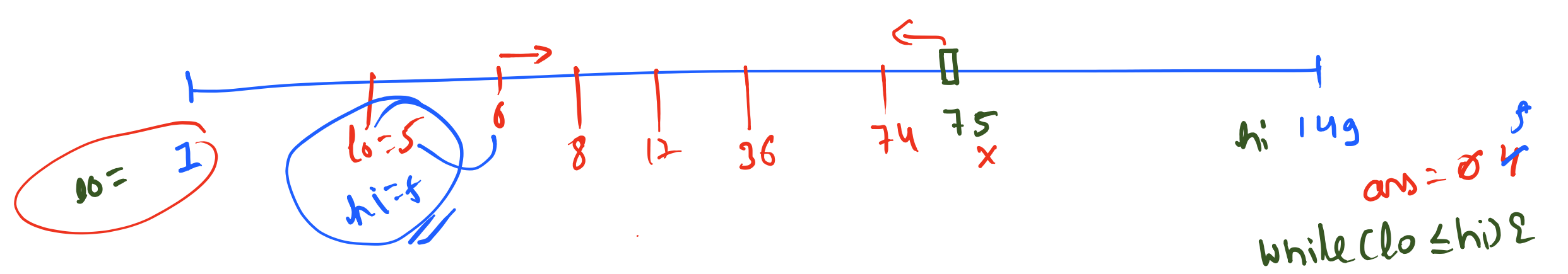
x=6 $6^3 \le 149$ X

```
x=1
while ( x^K <= mid ) {
    x++
}
return (x-1)
```

ans=mid

K=3
N=149

$75^3 \le 149$
$37^3 \le 149$ $18^3 \le 149$
$6^3 \le 149$ $64 \le 149$


lo= 1
lo=5
hi=5
6
8
12
36
74
75
hi 149

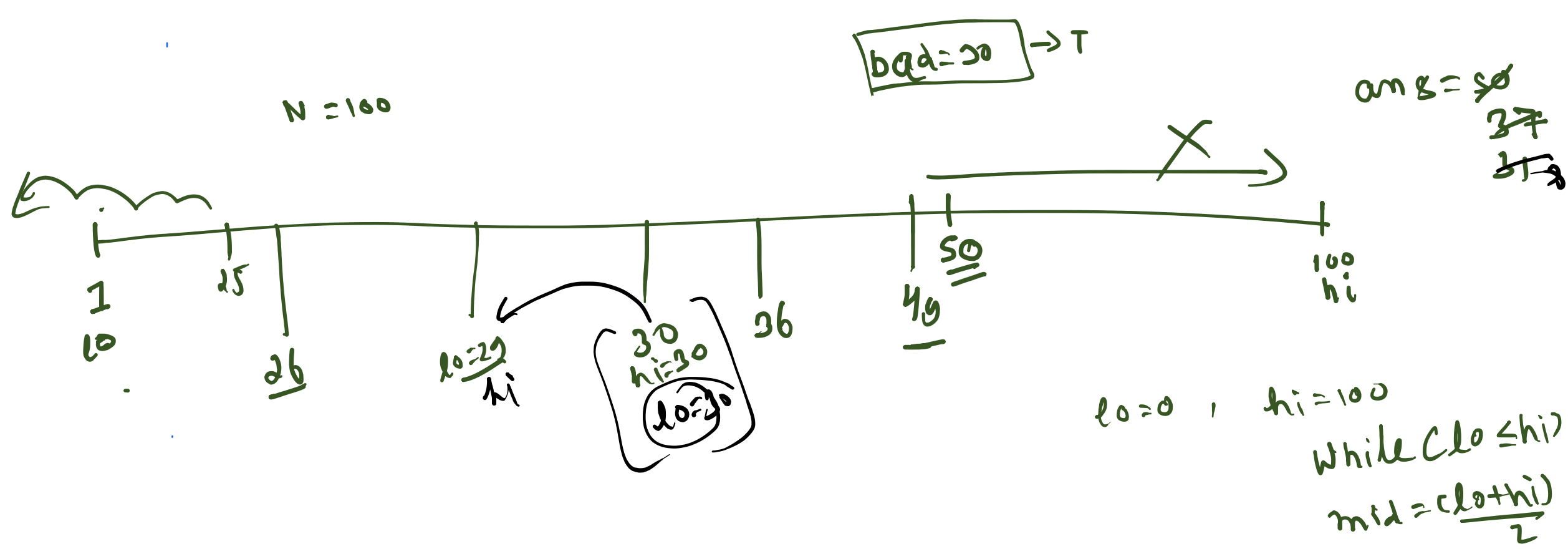
ans = ~~0~~ ~~4~~ 5

```
while (lo <= hi) {
    mid = (lo+hi)/2
    if ( mid^K <= N ) {
        ans=mid
        lo=mid+1
    }
    else
        hi=mid-1
    }
}
```

N =100

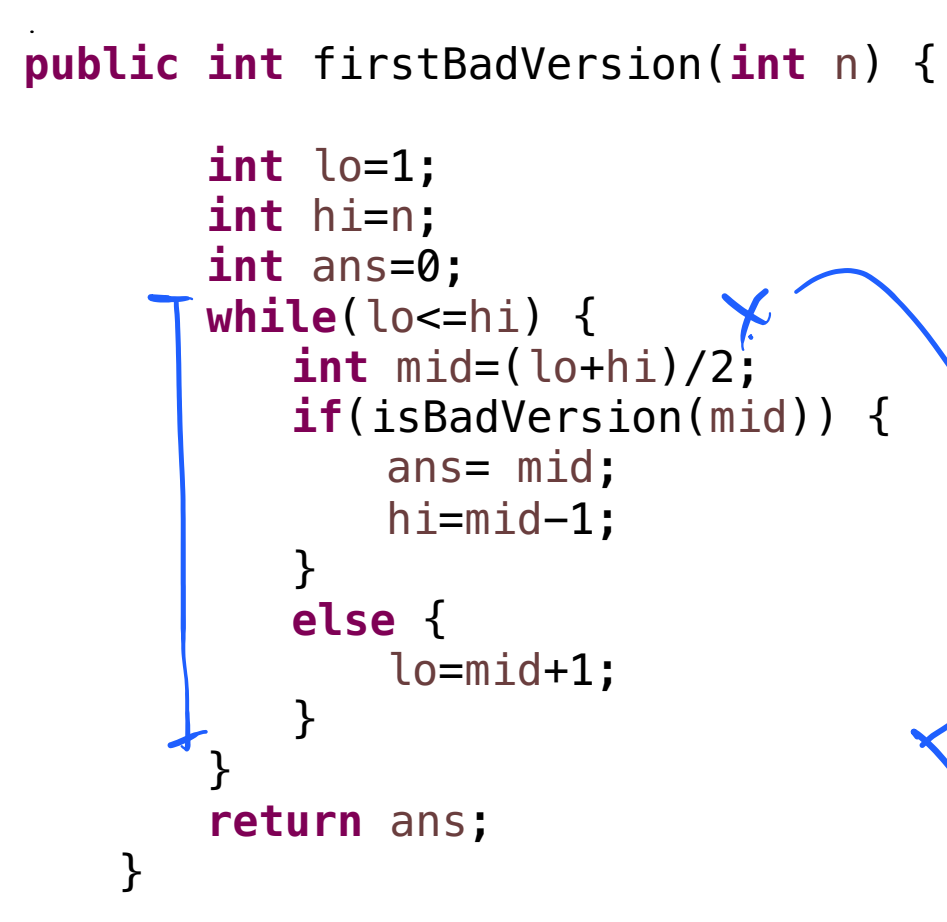
bad=30 -> T

ans = ~~50~~ ~~37~~ ~~31~~ 30


1
lo
25
26
lo=29
hi
30
hi=30
lo=30
36
49
50
100
hi

lo=0 , hi=100

```
while (lo <= hi) {
    mid = (lo+hi)/2
    if ( isbadversion(mid) ) {
        ans=mid
        hi=mid-1
    }
    else {
        lo=mid+1
    }
}
```



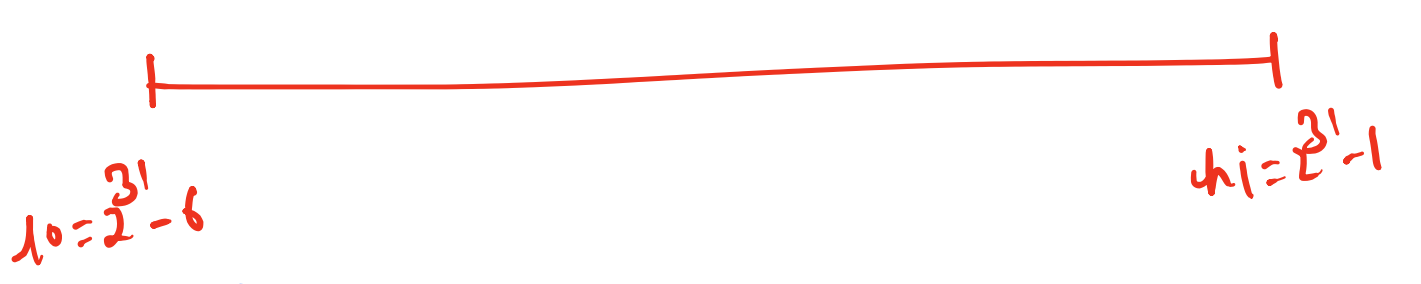
```java
public int firstBadVersion(int n) {

        int lo=1;
        int hi=n;
        int ans=0;
        while(lo<=hi) {
            int mid=(lo+hi)/2;
            if(isBadVersion(mid)) {
                ans= mid;
                hi=mid-1;
            }
            else {
                lo=mid+1;
            }
        }
        return ans;
    }
```

$lo + \frac{(hi-lo)}{2}$

$\frac{2 \times lo + hi - lo}{2}$

N = $2^{31}-1$ bad = $2^{31}-3$

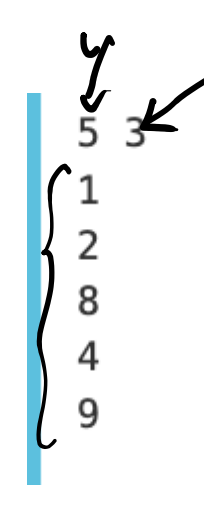

lo = $2^{31}-6$
hi = $2^{31}-1$

mid = $\frac{2^{31}-6+2^{31}-1}{2}$

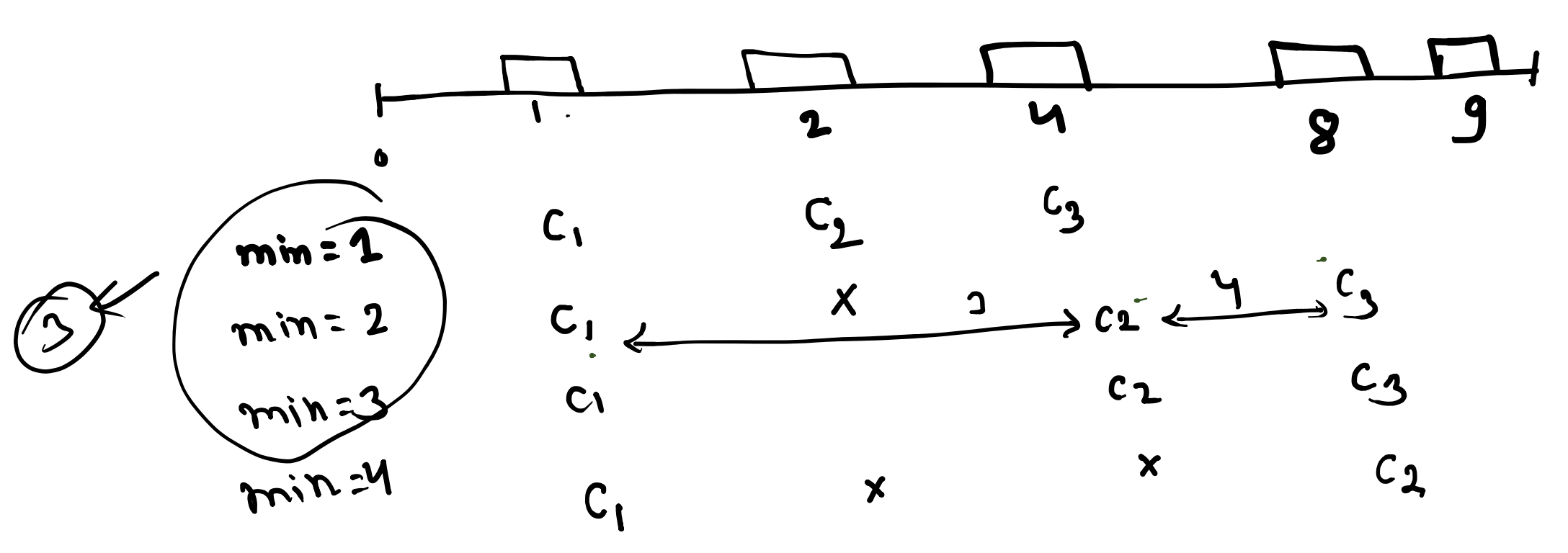
= $\frac{2 \times 2^{31} - 7}{2}$ -> Typecasting

= $2^{31}-3$

[1, 2, 8, 4, 9] -> [1, 2, 4, 8, 9] ✓



```
5 3
1
2
8
4
9
```


0
1
2
4
8
9
C1 C2 C3
C1 X 3 C2 4 C3
C1 C2 C3
C1 X X C2
min=1
min=2
min=3
min=4
3

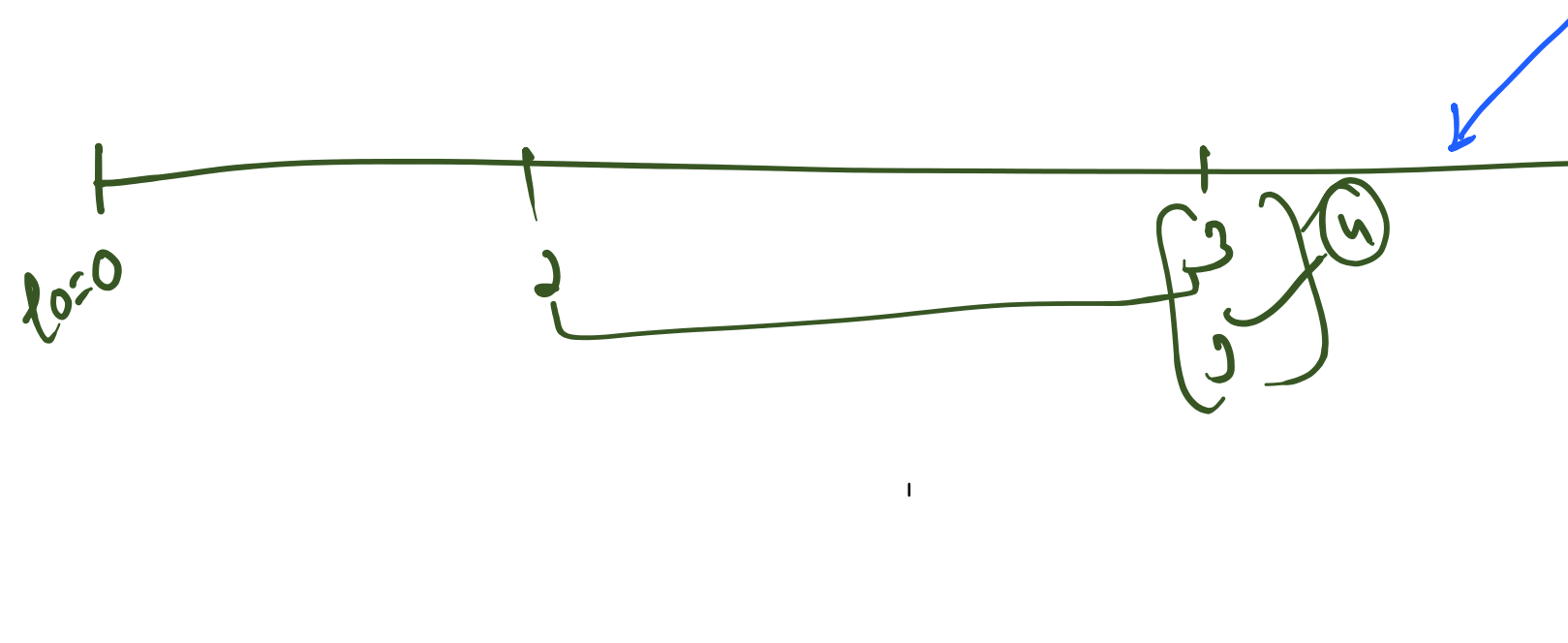
[1, 2, 4, 8, 9]

dis=~~1~~ ~~2~~ ~~3~~ 4

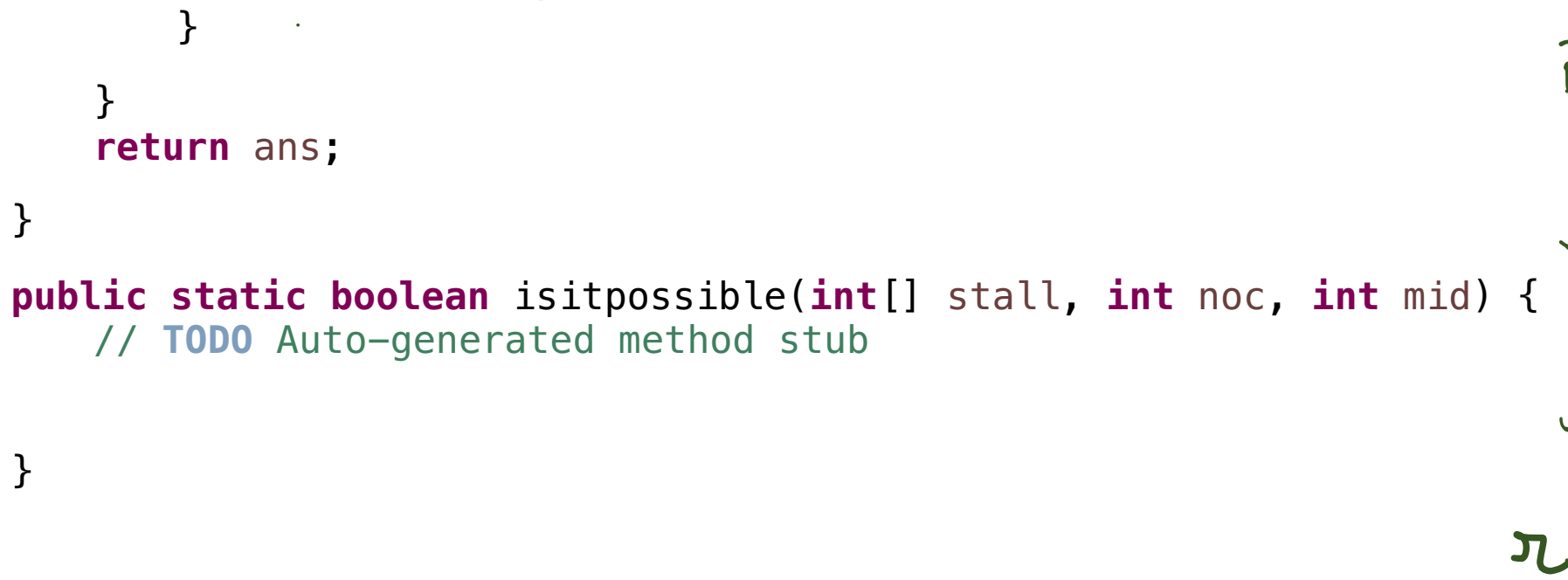
```
while (isitpossible(stall, noc, dis)) {
    dis++
}
return (dis-1)
```

[1, 2, 4, 8, 9]

ans=~~2~~ 3


distance
lene
lo=0
2
3
3
4
hi=8

```
while (lo <= hi) {
    mid=(lo+hi)/2
    if ( isitpossible(stall, noc, mid) ) {
        ans=mid
        lo=mid+1
    }
    else {
        hi=mid-1
    }
}
```



```java
public static int largest_distance(int[] stall, int noc) {
    int lo = 0;
    int hi = stall[stall.length - 1] - stall[0];
    int ans = 0;
    while (lo <= hi) {
        int mid = (lo + hi) / 2;
        if (isitpossible(stall, noc, mid)) {
            ans = mid;
            lo = mid + 1;
        } else {
            hi = mid - 1;
        }

    }
    return ans;
}
public static boolean isitpossible(int[] stall, int noc, int mid) {
    // TODO Auto-generated method stub

}
```

[1, 2, 4, 8, 9]
0 1 2 3 4

0 1 3

mid=1

```
cow=1
pos = stall[0]
for (i=1   i<stall.len i++) {
    if (stall[i]-pos) >= mid) {
        cow++
        pos = stall[i]
    }
    if ( cow == noc ) {
        return true
    }
}
return false
```